Packet-Level Network Inspection on Unrooted Android Using a Local VpnService Loopback

Capturing TCP, UDP and other IP traffic without root by routing the device through an on-device virtual network interface

Arunjith AS · sree.arunjith@gmail.com

White paper · Oct 2, 2026

Abstract

An unrooted Android device can capture its own TCP and UDP packets by acting as its own VPN. Android denies raw socket access to ordinary apps, so the usual tools (tcpdump, libpcap) need root. The VpnService API offers a supported alternative: an app declares itself the device VPN, receives every outbound IP packet on a file descriptor, and forwards the traffic onward itself. No remote VPN server is involved, and nothing leaves the device except the traffic that was always going to leave it.

This paper documents that design end to end. It covers the consent and interface setup, the routing loop that the design must avoid and the `protect()` call that avoids it, the userspace TCP state machine and UDP session table needed to relay traffic the kernel no longer handles, per-app attribution, and PCAP output that Wireshark can read.

The method's limits are structural rather than incidental. The capture sits at layer 3, so link-layer frames are never visible. Payloads above TCP remain encrypted. Only one VPN can be active at a time, and traffic from other devices on the same network is out of reach. The result is a complete view of what this device sends, not of what the network carries.

1. Introduction

The question this paper answers is how an ordinary Android app can see the device's own IP packets without root. Mobile users and researchers both need this. A user may want to know which servers an app contacts; a researcher may want to measure tracking, leakage or protocol behaviour across thousands of real devices. Neither can ask every participant to unlock a bootloader and flash a rooted image.

The obstacle is deliberate. Android's security model gives each app its own UID and denies it any view of other apps' sockets or of the kernel's packet path. A packet sniffer is, by definition, a request to see other apps' data, so the platform refuses it.

The VpnService API grants that view through a different door. A VPN app is expected to receive traffic, because that is what a VPN does. The platform hands it a TUN interface carrying every packet from every app, after an explicit user consent dialog that cannot be suppressed. An app that terminates this traffic locally rather than tunnelling it to a remote server gets a full on-device capture, using a public API, on stock firmware.

Contributions.

1. A description of the capture path from TUN file descriptor to stored packet, including the forwarding loop that naive implementations hit and the mechanism that prevents it.
2. The protocol handling that becomes the app's responsibility once the kernel's networking stack is bypassed, in particular a userspace TCP implementation.
3. An evaluation methodology that validates capture completeness against rooted `tcpdump` as ground truth.
4. A statement of the method's structural limits, separating what this approach cannot see from what merely needs more engineering.

2. Why packet capture is blocked

Four independent mechanisms stop an unprivileged Android app from sniffing packets, and defeating one does not help with the others.

No raw sockets. `AF_PACKET` and `SOCK_RAW` require `CAP_NET_RAW`, which ordinary app processes do not hold. Any `libpcap` build fails at the first `socket()` call. This is why every `tcpdump` guide for Android begins by telling the reader to root the device.

SELinux. Even a process that somehow obtained the capability is confined by the `untrusted_app` domain, which has no policy rule permitting raw packet sockets. SELinux has been enforcing on Android since 5.0, so this is not a check that can be bypassed from userspace.

No shared interface view. `/proc/net/tcp` and `/proc/net/udp` once let any app enumerate every socket on the device. Android 10 removed that read access as a privacy measure, so even connection metadata belonging to other apps is now hidden. Tools that mapped ports to apps by parsing those files broke at that release.

No promiscuous mode. Wi-Fi chipsets on consumer phones generally do not expose monitor mode, and enabling it would require driver-level access. Even with root, capturing other devices' frames is often impossible.

Taken together, these close every path that works on a desktop Linux machine. The VpnService route does not defeat any of them. It asks the platform for traffic through an interface built to deliver traffic, with the user's explicit consent, which is why it continues to work.

3. Related work

The local-VPN technique is roughly a decade old and has been validated at scale by several research systems, so this paper documents an established method rather than proposing a new one.

System	Year	Purpose	What it adds
PrivacyGuard	2015	Detect information leakage	First widely cited VPN-based interception platform; outperformed TaintDroid on leak detection
Haystack / Lumen	2015	In-situ traffic analysis	Adds a transparent MITM proxy for TLS traffic
AntMonitor	2015-2016	Large-scale packet measurement	Mobile-only design optimised for throughput and energy
MopEye	2017	Per-app network performance	Measures round-trip time opportunistically, without content inspection
PCAPdroid	current	User-facing capture tool	Open-source Android app; exports PCAP for Wireshark

Two findings from this literature shape the design decisions later in this paper.

The implementation determines the cost, not the method. AntMonitor reports reaching 94% of raw non-VPN throughput, while achieving 2x downlink and 8x uplink speeds against PrivacyGuard and Haystack, using 2-12x less energy. Those are implementations of the same API. The performance section of this paper therefore treats buffer handling and threading as first-order design choices.

Mobile-only beats client-server for privacy. Early designs routed traffic through a remote analysis server. AntMonitor's mobile-only architecture keeps packets on the device, which removes the operator as a party to the user's traffic. This paper assumes the mobile-only model for packet handling, with one deliberate departure: flow metadata is exported to a collector, as section 8 describes. That reintroduces the operator as a party to the data, which section 10 addresses.

The related risk is that the same API is widely abused. Free VPN apps on app stores use these interfaces to inspect and monetise user traffic, which is why the consent dialog and the limits in section 10 matter as much as the engineering.

4. The local VPN method

The app becomes the device's VPN but terminates the tunnel in its own process. Four steps set this up.

Declare the service. A service subclassing `VpnService` is declared in the manifest with the `android.permission.BIND_VPN_SERVICE` permission and the `android.net.VpnService` intent filter. The permission is held by the system, not requested from the user.

Obtain consent. `VpnService.prepare()` returns an Intent when the user has not yet approved this app as a VPN. The resulting system dialog warns that the app can intercept all network traffic, and it cannot be skipped, styled or auto-confirmed. A persistent key icon stays in the status bar while the VPN is up.

Build the interface. The `Builder` configures a TUN interface and returns a `ParcelFileDescriptor`:

```
ParcelFileDescriptor tun = new Builder()
    .setMtu(1500)
    .addAddress("10.215.173.1", 32)      // address of the virtual interface
    .addRoute("0.0.0.0", 0)              // claim all IPv4 traffic
    .addRoute("::", 0)                   // and all IPv6
    .addDnsServer("8.8.8.8")
    .addDisallowedApplication(getPackageName())
    .establish();
```

The `0.0.0.0/0` route is what makes this a capture: every packet from every app is delivered to this descriptor instead of the physical interface. The address is private to the virtual interface and is never seen on the network.

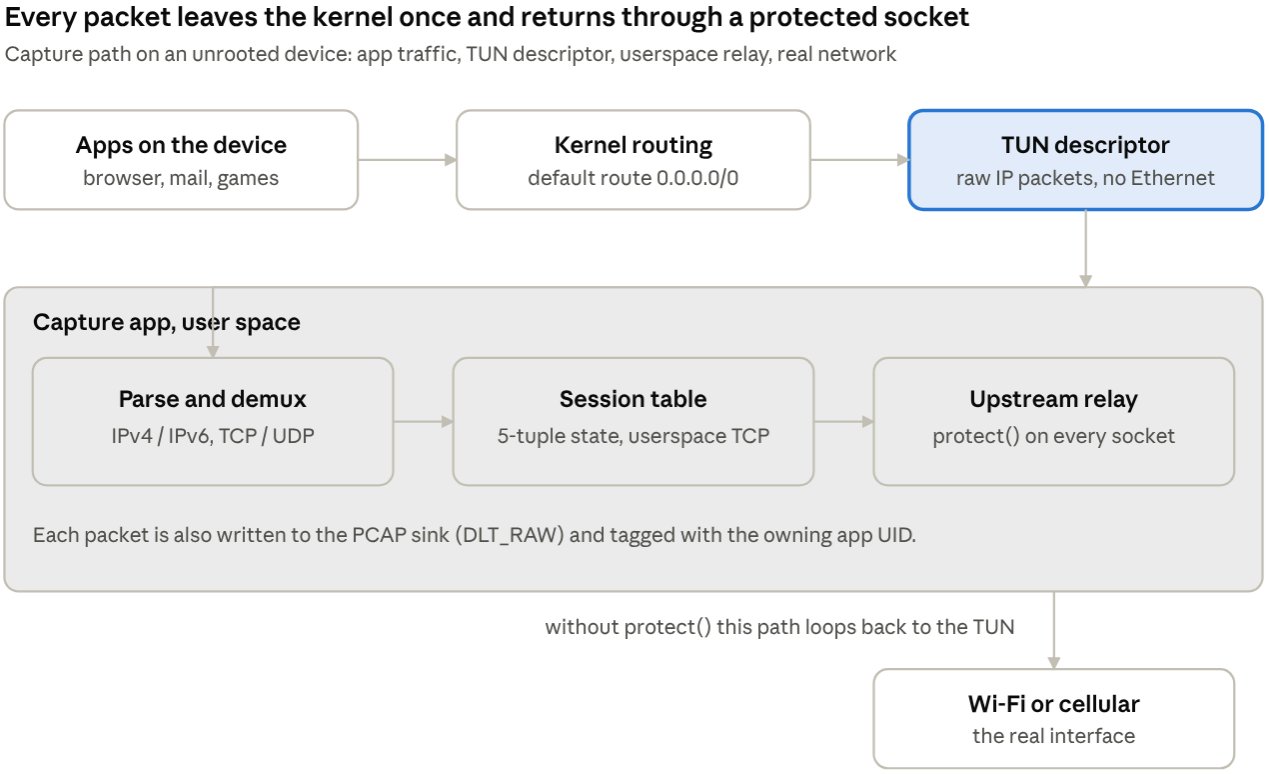
Break the loop. This is the step that defeats naive implementations. Once the default route points at the TUN interface, the app's own upstream sockets are also routed into the TUN, so packets the app sends to forward traffic are handed straight back to the app. The fix is `VpnService.protect(socket)`, which marks a socket to bypass the VPN and use the real interface. Every upstream socket must be protected immediately after creation and before it connects.

The `addDisallowedApplication` call excludes the capture app's own traffic from the tunnel, which helps but does not replace `protect()`: the exclusion applies at the routing layer and does not cover sockets opened before the VPN is established.

What arrives on the descriptor is a raw IP packet, starting at the IP header. There is no Ethernet header, because a TUN interface operates at layer 3. This single fact determines both the PCAP link type in section 7 and several of the limits in section 10.

5. System architecture

The pipeline is a loop with four stages, and the device's real network sits outside it.



capture path · 4 stages, 1 loop hazard

The capture thread blocks on a read from the TUN descriptor, parses each packet, looks up or creates a session, and hands the payload to a protected upstream socket. A second thread polls those sockets for inbound data, synthesises the IP and transport headers for the reply, and writes it back into the descriptor, where the kernel delivers it to the originating app as though it came from the network.

Three design choices decide whether the result is usable:

Threading. Reading, writing and upstream I/O must not share a thread. A single-threaded loop stalls the whole device's traffic whenever one upstream socket is slow. Use non-blocking channels with a selector for the upstream side.

Buffer discipline. The TUN read allocates per packet unless buffers are pooled, and at a few thousand packets per second that allocation rate dominates the profile. Reuse direct byte buffers sized to the MTU.

Session lifetime. Every entry in the table holds a real socket. Without idle timeouts and explicit teardown on FIN or RST, file descriptors leak and the app eventually fails to open new connections.

6. Protocol handling

Because the app receives IP packets rather than socket data, it must implement the transport layer itself. This is the largest engineering cost of the method, and it falls almost entirely on TCP.

IP demultiplexing. Read the version field in the first nibble, then the protocol field: 6 for TCP, 17 for UDP, 1 for ICMP, 58 for ICMPv6. Validate the header checksum, handle IPv4 options, and handle IPv6 extension header chains before the transport header is reachable. Fragmented packets must be reassembled before the transport header can be parsed at all.

UDP is straightforward. Each unique 5-tuple gets a `DatagramChannel`, protected, connected to the destination. Outbound payloads are written to it; inbound datagrams are wrapped in a synthesised IP and UDP header with the addresses reversed and written back to the TUN descriptor. Sessions are dropped after an idle timeout, typically 30 to 60 seconds.

TCP requires a userspace stack. The app sees SYN, ACK and FIN flags, sequence numbers and window fields, and the peer expects correct responses to all of them. The app must:

- Answer the client's SYN with a synthesised SYN-ACK, having first opened and connected the real upstream socket.
- Maintain sequence and acknowledgement numbers in both directions, with independent initial sequence numbers.
- Advertise a receive window and honour the peer's, applying backpressure when the upstream socket is slower than the app is sending.
- Retransmit unacknowledged data on a timer.
- Walk the connection through FIN-WAIT, CLOSE-WAIT, TIME-WAIT and RST correctly, or sockets leak.

Implementing this from scratch is a well-known source of subtle bugs. The practical options are to embed a small TCP/IP stack such as lwIP, or to write a half-stack that terminates the client connection and relays bytes over a normal `SocketChannel`. The

second is simpler, and it is sufficient when the goal is capture rather than modification.

ICMP is largely out of reach. Forwarding a ping requires a raw socket, which is exactly what the platform denies. Some implementations answer echo requests locally, which is a useful fiction but means `ping` results in the capture do not reflect the real network.

DNS deserves separate handling. Queries on port 53 are the cheapest high-value signal in the whole capture, since they name the destination before any encryption begins. Parsing them is also what lets later TLS flows be labelled with a hostname rather than an IP address.

7. Output, attribution and analysis

Captured packets are written as PCAP so that existing tooling applies unchanged.

Link type. The file header must declare `DLT_RAW` (link type 101), not Ethernet. Packets from a TUN interface begin at the IP header, and a file that claims Ethernet makes Wireshark misparse every frame by 14 bytes. PCAPNG is the better choice when per-packet comments are wanted, since the app name can be attached to each record.

Timestamps. Use a monotonic clock for intervals and the wall clock only for the file's absolute times. Capture timestamps are taken when the packet is read from the descriptor, which is already later than when the originating app sent it; this offset matters for latency measurements and should be stated in any paper reporting timing results.

Per-app attribution. This is the capability a desktop capture does not have, and it is often the main reason to capture on the device. Android 10 and later expose `ConnectivityManager.getConnectionOwnerUid()`, which maps a 5-tuple to the owning UID, which `PackageManager` then resolves to a package name. The lookup must happen while the connection is alive, so it belongs in the session-creation path rather than in post-processing. On older releases the same mapping came from `/proc/net/tcp`, which is no longer readable.

Encrypted traffic. Most flows will be TLS, so payloads are opaque. What remains visible is still substantial: destination IP and port, timing and packet sizes, DNS queries, and the Server Name Indication field in the TLS ClientHello, which carries the hostname in cleartext for most connections.

Decryption is possible in principle by installing a user CA certificate and running a MITM proxy, as Haystack does. It is poorly suited to field research on modern Android. Since Android 7, apps do not trust user-installed CAs unless their network security configuration opts in, and any app using certificate pinning will fail regardless. A study that depends on cleartext payloads should expect to cover only a minority of apps, and must obtain informed consent to intercept them.

8. Flow export to a remote collector

Storing every packet on the device is impractical for a study of any length, so the system summarises each session as a flow record and exports the metadata to a collector. The capture handles IPv4 and IPv6 equally, and the export format has to preserve that distinction rather than flatten it.

IPFIX rather than NetFlow v5. NetFlow v5 has a fixed record layout and no IPv6 addresses at all, which rules it out here. IPFIX (RFC 7011) is template-based: the exporter defines the record shape, and enterprise-specific elements carry the fields that make on-device capture valuable, namely the owning app, the DNS query name and the TLS server name. NetFlow v9 is the fallback where a collector cannot speak IPFIX.

Two templates, one record type. Addresses differ in width between the protocols, so the exporter defines an IPv4 template and an IPv6 template and selects between them per flow. Every other element is shared.

Field	IPFIX element	Notes
IP version	ipVersion (60)	4 or 6; lets the collector group without inspecting addresses
Source address	sourceIPv4Address (8) / sourceIPv6Address (27)	the per-template difference
Destination address	destinationIPv4Address (12) / destinationIPv6Address (28)	as above
Ports	sourceTransportPort (7), destinationTransportPort (11)	identical in both
Protocol	protocolIdentifier (4)	6, 17, 1 or 58
Bytes and packets	octetDeltaCount (1), packetDeltaCount (2)	plus reverse elements for the return direction
Timing	flowStartMilliseconds (152), flowEndMilliseconds (153)	millisecond precision is enough for flow analysis
TCP flags	tcpControlBits (6)	union over the flow; separates a refused connection from an idle one
IPv6 flow label	flowLabelIPv6 (31)	IPv6 only; omitted from the IPv4 template
App identity	enterprise-specific	package name and UID from section 7

Field	IPFIX element	Notes
Names	enterprise-specific	DNS query name and TLS SNI

Bidirectional records. The capture sees both halves of every session, so each flow is exported once as a biflow (RFC 5103) with reverse counters, rather than as two unidirectional records the collector must pair up. This halves record volume and removes a class of stitching errors.

What dual-stack changes in practice. Four behaviours affect the data and belong in the exporter rather than being left to the analyst:

- **Happy Eyeballs (RFC 8305).** A single user action produces parallel A and AAAA connection attempts, one of which is abandoned. Flow counts inflate unless records are grouped by hostname, and short zero-byte flows are a normal artefact rather than a failure.
- **Privacy addresses (RFC 8981).** The device's own IPv6 source address rotates periodically, so it cannot serve as a stable identifier across a long study. Key on app identity instead.
- **NAT64 and 464XLAT.** Many mobile networks are IPv6-only with translation, and Android's CLAT presents IPv4 to apps while the network carries IPv6. A flow may therefore appear as IPv4 at the TUN interface although no IPv4 packet reached the radio. Any claim about IPv6 adoption measured this way needs that caveat stated.
- **Local-scope traffic.** Link-local and multicast destinations (fe80::/10, ff02::/16) are routine and never leave the device. Classify them rather than exporting them as ordinary flows.

Transport and timeouts. Classic NetFlow ships over UDP on the assumption of a reliable LAN. A mobile network loses those records silently, so batches go over HTTPS or IPFIX-over-TLS with on-disk buffering, which means a process kill costs nothing. Active and idle timeouts of 60 and 15 seconds are reasonable starting points; longer active timeouts give fewer records and fewer radio wakeups at the cost of coarser timing.

Protect the exporter's own socket. The upload socket must be passed to `protect()` like any other. Without it, uploads route into the TUN, generate flow records describing themselves, and feed a loop that grows with every flush. The same applies to any DNS lookup made to resolve the collector.

Volume. A phone generates a few thousand flows a day, so a day of metadata is a few hundred kilobytes. That is small enough to batch for the radio rather than stream, and small enough that a week of buffering fits comfortably on the device.

9. Evaluation methodology

A capture tool must be shown to be both complete and cheap. Completeness is the harder claim, because a tool that silently drops packets looks identical to a quiet network.

Ground truth. Run the capture app on a rooted device while `tcpdump` records the physical interface simultaneously. Every packet the app records should appear in the `tcpdump` trace, and the reverse comparison gives the miss rate. Report the miss rate per protocol, since TCP retransmission can mask losses that UDP exposes immediately.

Throughput. Measure with `iperf3` against a local server in four conditions: no VPN, the capture app in pass-through mode, the app writing PCAP, and the app performing inspection. The no-VPN number is the ceiling; AntMonitor's reported 94% of raw throughput is a reasonable target to compare against.

Latency. Report added round-trip time at the median and the 95th percentile. A median is not enough, because userspace TCP relaying produces a long tail under load.

CPU and memory. Sample during sustained transfer, not at idle. Memory growth over a long run is the test that exposes leaked sessions in the state table.

Energy. Use an external power monitor rather than Android's battery statistics, which attribute all tunnelled traffic to the VPN app and so cannot separate the tool's own cost.

Workload. Use a scripted app interaction so runs are comparable, and report device model, Android version, network type and the number of concurrent flows. Results from an idle phone on Wi-Fi do not generalise to a loaded phone on a cellular network.

Two controls are worth building in. Verify that the capture app's own upstream traffic is absent from the output, which confirms `protect()` is applied everywhere. Verify that the session table returns to zero entries after an idle period, which confirms timeouts work.

10. Limitations and responsible use

Some of these limits can be engineered away and some cannot. The distinction matters for any paper making claims about coverage.

Structural limits, which no implementation can remove:

- **No link layer.** A TUN interface delivers IP packets, so Ethernet and Wi-Fi frames, ARP and DHCP are invisible. Anything below layer 3 is out of scope.
- **This device only.** Traffic from other devices on the same network is unreachable, as is traffic routed through the phone while tethering.

- **One VPN at a time.** Android permits a single active VPN, so the capture tool cannot coexist with a VPN the user relies on for work or privacy. This is the most common reason participants drop out of field studies.
- **Encrypted payloads.** Discussed in section 7; cleartext content is the exception, not the rule.
- **Some system traffic bypasses the tunnel.** Connectivity checks and certain system components are not subject to the VPN route, so a capture is not a complete record of the device's radio activity.

Engineering limits, which better implementations reduce:

- Throughput and latency overhead from userspace relaying.
- Battery drain, which also risks the app being killed by aggressive OEM power management; a foreground service and a battery-optimisation exemption are usually required for long runs.
- Correctness gaps in the TCP implementation under packet loss, reordering or rapid connection churn.

Legal and ethical constraints. Capturing traffic is lawful on a device you own, and research use needs informed consent plus ethics board approval. The consent must be specific: participants should understand that a tool with this access can see every destination their phone contacts, and MITM interception requires separate and explicit agreement.

Exported metadata is still personal data. Flow records name every destination a participant's phone contacts, with timestamps and the app responsible. That is a behavioural record, and in some cases it reveals more than payloads would. Four commitments should be written into the study design and stated in the paper: what is exported, how long it is retained, who can query the collector, and what is truncated or hashed before it leaves the device. Hostnames and destination addresses are the fields that most often need coarsening. Under GDPR and comparable regimes this processing needs a lawful basis and a named controller, and the collector's own location determines where the data sits.

Distribution. Google Play treats VpnService as a sensitive API. Apps using it must declare it, must use it for a core user-facing function rather than as a means to collect data, and must disclose the behaviour prominently. A research app is usually better distributed directly or through a managed study channel than through the public store.

11. Conclusion

The local VpnService loopback converts a platform restriction into a tractable engineering problem. Android will not let an app read other apps' packets, but it will hand an app every packet the device sends if the app agrees to be responsible for delivering them. Accepting that responsibility means writing a userspace transport layer, which is the real cost of the method.

What the technique yields is a complete, app-attributed, layer-3 view of one device's traffic on stock firmware, with user consent, using public API. What it does not yield is a network view: no frames, no neighbouring devices, and little cleartext above TCP.

Flow export extends the method from a single-device tool to a measurement platform, at the cost of making the operator a holder of participants' metadata. The open work is mostly in protocol coverage and cost. QUIC now carries a large share of mobile traffic as UDP on port 443, which makes the UDP path, long treated as the simple case, the one that matters most. Energy remains the binding constraint on long field deployments.

References

[1] Y. Song and U. Hengartner. PrivacyGuard: A VPN-based platform to detect information leakage on Android devices. In *Proceedings of the 5th Annual ACM CCS Workshop on Security and Privacy in Smartphones and Mobile Devices (SPSM)*, pages 15-26, 2015.

[2] A. Razaghpanah, N. Vallina-Rodriguez, S. Sundaresan, C. Kreibich, P. Gill, M. Allman and V. Paxson. Haystack: In situ mobile traffic analysis in user space. arXiv:1510.01419, October 2015. Later revised as *Haystack: A multi-purpose mobile vantage point in user space*.

[3] A. Le, J. Varmarken, S. Langhoff, A. Shuba, M. Gjoka and A. Markopoulou. AntMonitor: A system for monitoring from mobile devices. In *Proceedings of the 2015 ACM SIGCOMM Workshop on Crowdsourcing and Crowdsharing of Big (Internet) Data (C2B1D)*, pages 15-20, 2015.

[4] A. Shuba, A. Le, E. Alimpertis, M. Gjoka and A. Markopoulou. AntMonitor: System and applications. arXiv:1611.04268, 2016. doi:10.48550/arXiv.1611.04268

[5] D. Wu, R. K. C. Chang, W. Li, E. K. T. Cheng and D. Gao. MopEye: Opportunistic monitoring of per-app mobile network performance. In *Proceedings of the 2017 USENIX Annual Technical Conference (ATC)*, 2017.

[6] J. Ren, A. Rao, M. Lindorfer, A. Legout and D. Choffnes. ReCon: Revealing and controlling PII leaks in mobile network traffic. In *Proceedings of the 14th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2016.

[7] E. Papadogiannaki and S. Ioannidis. A survey on encrypted network traffic analysis applications, techniques, and countermeasures. *ACM Computing Surveys*, 54(6), 2021. doi:10.1145/3457904

[8] Android Developers. *VpnService* API reference and *Android 10 privacy changes*. Google, current documentation.

[9] PCAPdroid. Open-source Android packet capture application using VpnService. Source repository and documentation.

Verification note. Entries [1] to [7] were checked against publisher records, author listings or the papers themselves; page ranges for [1] and [3] come from citing bibliographies rather than the publisher page, so confirm them against the ACM Digital Library before submission. Entries [8] and [9] are living documents and should be cited with the access date.